A Cost-Aware Approach for Collaborating Large Language Models and Small Language Models

Zheng Li
State Key Laboratory of Novel Software Technology
Nanjing University
Nanjing, Jiangsu, China
zheng_li@smail.nju.edu.cn

Xuyun Zhang
School of Computing
Macquarie University
Sidney, NSW, Australia
xuyun.zhang@mq.edu.cn

Sheng Lu
State Key Laboratory of Novel Software Technology
Nanjing University
Nanjing, Jiangsu, China
sheng-lu@smail.nju.edu.cn

Hua Deng
State Key Laboratory of Novel Software Technology
Nanjing University
Nanjing, Jiangsu, China
hdeng@smail.nju.edu.cn

Hao Tian
State Key Laboratory of Novel Software Technology
Nanjing University
Nanjing, Jiangsu, China
htian@smail.nju.edu.cn

Wanchun Dou
State Key Laboratory of Novel Software Technology
Nanjing University
Nanjing, Jiangsu, China
douwc@nju.edu.cn

Abstract

The emerging reasoning ability of large language models (LLMs) and accompanying commercial applications offer a promising path for service providers to deploy intelligent agents on their own products through API calls. However, the black-box nature of LLMs has driven providers to try prompt tuning to improve reasoning quality for competitiveness, while the generated reasoning logic results in additional service costs. Although some works have proposed collaborating LLMs and Small Language Models (SLMs) to reduce the frequency of LLM calls, most overlook the actual number of tokens interacting with the LLMs, which results in potentially high costs still. Furthermore, directly compressing the prompt to reduce tokens often leads to a significant accuracy loss. To address the above challenges, we propose a cost-aware approach for collaborating LLMs and SLMs, named Coco. In our method, a confidence-based task assignment method is designed which leverages the result confidence of SLMs to assess task complexity and determine whether LLM involvement is necessary. For complex tasks, the SLM adapts the input by compressing unnecessary information according to confidence. Considering the potential loss of accuracy, prompt tuning-based reasoning optimization methods are introduced to guide the LLM in generating both the reasoning logic sketch and the final result. Finally, logic alignment is applied to fuse sketches from both models, ensuring the rationality of the reasoning logic. Experimental results on three open-source datasets demonstrate that our approach effectively reduces the cost of API calls to LLMs while ensuring the reasoning accuracy and the reasonableness of generated logic.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**.

Keywords

Large and Small Language Models, Task Assignment, Cost-aware Collaboration, Prompt Tuning, Reasoning Optimization

ACM Reference Format:
Zheng Li, Xuyun Zhang, Sheng Lu, Hua Deng, Hao Tian, and Wanchun Dou. 2025. A Cost-Aware Approach for Collaborating Large Language Models and Small Language Models. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25), November 10–14, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3746252.3760990

1 Introduction

Pre-trained Large Language Models (LLMs) have shown significant advantages in natural language processing by learning complex language patterns from vast datasets. These models excel in generalizing across a wide range of tasks, requiring minimal task-specific adaptation through few-shot learning [18]. Compared to traditional models, pre-trained LLMs demonstrate superior language understanding and generation capabilities, making them fundamental in applications such as text generation, translation, and reasoning [7, 12, 30]. With the increasing adoption of LLMs and the gradual maturation of their commercial ecosystems, an increasing number of smart devices and software products are integrating these models to provide more intelligent and efficient downstream services. However, most product teams lack the expertise or resources to deploy large-scale models such as ChatGPT or DeepSeek, relying instead on paid API calls to act as intermediaries in delivering services to users. The service fee is calculated based on the number of input and output tokens and their respective unit prices. Given the current rate of $150 per million output tokens for the latest

Wanchun Dou is the corresponding author.

CIKM '25, Seoul, Republic of Korea.

ACM ISBN 979-8-4007-2040-6/2025/11
https://doi.org/10.1145/3746252.3760990

GPT-4.5, the cost of bulk services remains prohibitively high for intermediate product teams.

In addition, API-based interactions with LLMs operate as black-box processes, preventing developers from modifying or optimizing the underlying model architecture and parameters [5, 23]. This limitation presents a major challenge in enhancing reasoning accuracy, which is crucial for improving the competitiveness of derivative services. To address this issue, various reasoning optimization techniques based on prompt engineering have been extensively studied [2, 11, 24]. Among them, Chain of Thought (CoT) Prompting has gained widespread recognition, which guides the model through step-by-step reasoning by structuring prompts in a way that encourages logical decomposition during response generation, thereby improving both the accuracy and interpretability of the output [28]. However, as illustrated in Fig. 1, the additional reasoning steps often generate several times more tokens than the primary output, and the high cost per token can lead to rapidly escalating service expenses.

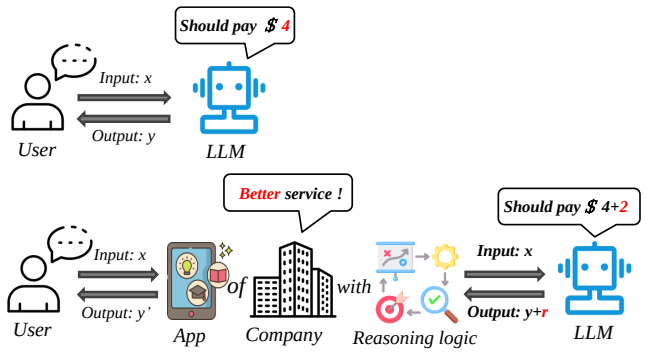


Figure 1: Illustration of additional service costs caused by the chain of thought.

In such a scenario, prompt compression methods, such as prompt summarization and pruning, are initially considered to reduce the token count of inputs to LLMs [10, 25]. Unfortunately, these methods are unable to compress the output content of LLMs directly and are prone to over-compression when dealing with logical reasoning tasks because of insufficient consideration of the complexity of the reasoning task, resulting in a negative impact on the reasoning accuracy of LLMs [19]. Recently, some researchers have shifted their focus to the paradigm of locally deploying Small Language Models (SLMs) in collaboration with LLMs. On the one hand, the operational cost of small models is virtually negligible [6]. On the other hand, fine-tuning these small models with domain-specific data enables them to acquire richer domain knowledge, compensating for the potential lack of attention to localized knowledge by LLMs [8, 14]. By constructing an effective collaboration mechanism, this approach has the potential to reduce reliance on LLM API calls, thereby controlling service costs. However, existing approaches for SLMs and LLMs collaboration still have certain limitations in cost reduction. For instance, some research suggests reserving simple tasks for local processing to reduce LLM calls. However, this overlooks the fact that complex tasks often require lengthy and intricate contexts, leading to a high number of tokens still being input to the LLMs.

With these observations, it is still a challenge to reduce the token interaction between LLMs and SLMs, while ensuring inference accuracy. In view of this challenge, a cost-aware approach for collaborating LLMs and SLMs is proposed in this paper, termed Coco, which consists of confidence-based task assignment and prompt tuning-based reasoning optimization. Specifically, when a logical reasoning task is received, the SLM is prompted to generate an initial reasoning logic and the final result. Meanwhile, the confidence is computed using the logits of output tokens, which helps assess the task complexity and decide whether to involve the LLMs. For complex tasks, SLMs adaptively compress the input based on the highest probability value from the logits, reducing the amount of original task information passed to LLMs. In the prompt design for LLMs, the SLMs provide the least likely reasoning results, guiding the LLMs to output only the final reasoning result and the corresponding sketch. Semantic alignment is deployed to fuse the reasoning sketch and logic, providing guidance to SLMs for generating the final output and completing the reasoning process. Overall, the main contributions of our work can be summarized as follows:

- We propose a cost-aware approach for collaborating LLMs and SLMs which effectively reduces the frequency of LLM calls while further reducing the number of tokens interacting with the LLMs.
- We introduce a confidence-based task assignment method and prompt tuning-based reasoning optimization techniques, which ensures the collaborative reasoning accuracy while reducing the reasoning cost.
- Experimental results on three open-source datasets demonstrate the effectiveness of our method.

The remainder of the paper is organized as follows. The related work is reviewed in Section 2. Section 3 presents the details of our proposed method. Extensive experiments are conducted, and detailed analyses are illustrated in Section 4. Finally, Section 5 addresses our conclusions and future work.

2 Related works

Our work aims to reduce the service cost of logical reasoning tasks by collaborating LLMs and SLMs. Therefore, we review methods for the collaboration of LLMs and SLMs and summarize related prompt tuning approaches.

Collaboration of LLMs and SLMs. The collaboration between LLMs and SLMs seeks to balance the computational efficiency of SLMs with the powerful reasoning capabilities of LLMs. LLMs excel at complex tasks but are resource-intensive. SLMs, while efficient and cost-effective, often struggle to solve more challenging problems.

To compensate for the limitations of a single model, recent studies have shown that with appropriate prompting strategies, LLMs can effectively complement SLMs, handling complex samples that are difficult for SLMs [9]. For example, some studies propose a filter-then-rerank framework, where SLMs filter the samples, and LLMs rerank the challenging samples, significantly improving task performance [16]. Additionally, given the high computational cost

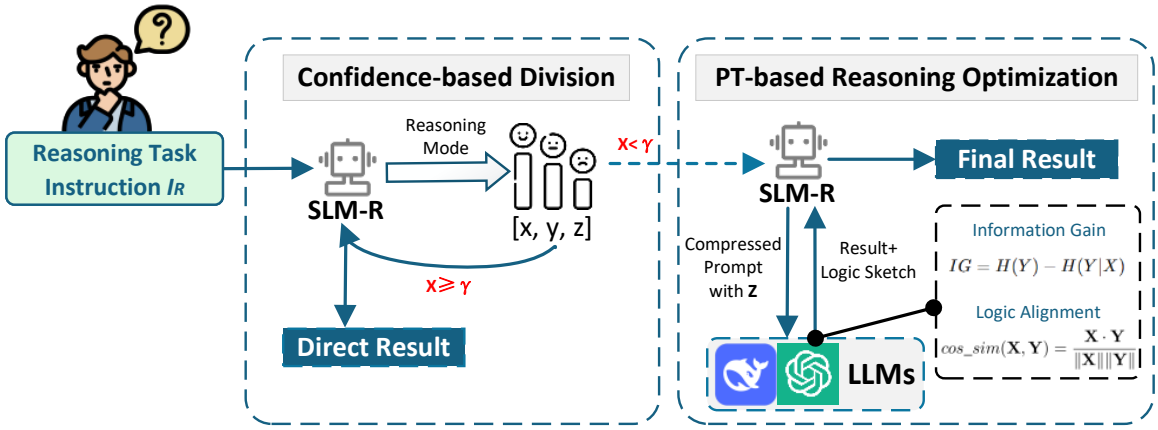


Figure 2: The framework of the cost-aware approach for collaborating LLMs and SLMs.

of LLMs in generation tasks, some research has introduced hybrid model architectures that combine LLMs and SLMs to enhance autoregressive decoding efficiency. Experiments show that this method achieves up to 4× speedup while incurring minimal performance loss [4]. Furthermore, to further improve the performance of small models, another study proposes a step-by-step distillation mechanism, which trains small models by extracting knowledge from the LLM reasoning process, allowing them to outperform LLMs with less training data [13]. With the growing need for privacy protection, the CoGenesis framework was also introduced, integrating cloud-hosted LLMs with locally deployed small SLMs, thus avoiding the need to upload user data and ensuring privacy [32].

Prompt Tuning. Prompt tuning is a method that adjusts the output of pre-trained language models by optimizing input prompts, aiming to improve performance on specific tasks. By designing effective prompts, this approach enhances performance without the need for extensive parameter adjustments, making it efficient and requiring fewer computational resources [31]. Mainstream prompt tuning methods include manual prompting, prefix tuning, prompt injection, and soft prompting, all of which enhance model performance with minimal parameter adjustments and computational resources [15] [26].

The Chain-of-Thought prompting, as a widely recognized method of prompt tuning, has significantly improved reasoning in LLMs by prompting them to generate intermediate reasoning steps [28]. This method enhances performance on tasks requiring multi-step reasoning, such as mathematical and commonsense reasoning. However, follow-up research indicates that the reasoning paths are not always optimal, often lacking deliberation and efficiency [35] [22]. Several variants have been proposed to address these limitations. Self-Consistency generates multiple reasoning chains and selects the most consistent result, improving robustness [27]. The Tree-of-Thought (ToT) extends CoT by employing tree-searching to explore the reasoning space more thoroughly, identifying better reasoning paths that CoT might miss, but at the cost of increased inference complexity [29]. To overcome this, Chain of Preference Optimization fine-tunes LLMs to align CoT reasoning paths with the tree-searching outcomes of ToT, maintaining the benefits of ToT without the associated computational burden [33]. These advancements collectively refine CoT, enhancing reasoning performance while improving efficiency.

3 A Cost-aware Approach for Collaborating LLMs and SLMs

3.1 Method Overview

In this work, we propose a cost-aware approach for collaborating LLMs and SLMs, named Coco. By leveraging the strengths of both models, the negligible running costs and domain-specific knowledge of SLMs, and the strong generalization capabilities of LLMs, Coco aims to reduce the service cost by reducing the number of tokens interacted between SLMs and LLMs, while maintaining the accuracy of the reasoning. As illustrated in Fig. 2, Coco consists of two core parts, confidence-based task assignment and prompt tuning (PT)-based reasoning optimization, together contributing to efficient task reasoning and cost reduction.

In the task assignment phase, a prefabricated prompt is provided to a fine-tuned SLM upon receiving a logical reasoning task, prompting it to first output the logical sketch of the reasoning and then the final reasoning result. The complexity of the task is assessed using the confidence calculated from the logits of the output tokens. If the task complexity is low, the SLM processes the task entirely. For more complex tasks, LLMs will be involved in collaborative reasoning. LLM then receives another prefabricated prompt containing the compressed problem as well as what SLM considers to be multiple unlikely results. For large models, in addition to outputting the results, a logical sketch of the reasoning is also output. Finally, semantic alignment is applied to fuse the outputs from both models. This fusion ensures that the reasoning process remains consistent and accurate, providing the SLM with guidance to refine its final output and complete the reasoning task.

3.2 Confidence-based Task Assignment

While the general concept of reducing costs by keeping simple tasks local and delegating complex ones to LLMs has been previously explored, our approach diverges significantly in both scope and

implementation. Rather than relying on lightweight classification models as in [6], we utilize SLMs locally—not only for basic classification, but also for generating logically structured responses. This extends the role of SLMs beyond simple decision-making, enabling more intelligent and flexible reasoning. A key challenge in language model reasoning lies in estimating the confidence of their outputs. Prior work has attempted to address this by prompting LLMs to output self-assessed confidence scores, followed by calibration techniques. However, these methods are less effective for SLMs, which are more prone to hallucination and tend to overstate their certainty, making their self-reported confidence values unreliable. For this reason, we propose a token logits-based probabilistic mapping method to obtain the confidence of reasoning results, which is illustrated in Fig. 3.

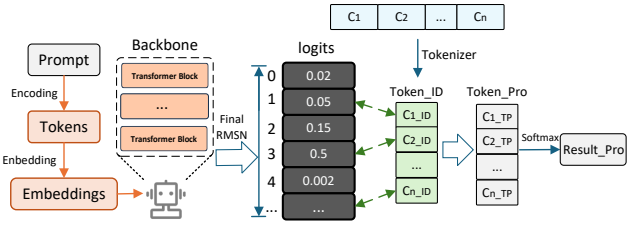


Figure 3: The illustration of token logits-based probabilistic mapping method.

The original input problem is defined as x, and the local SLM is denoted as M_s. To more reliably obtain the expected result probabilities in the SLM reasoning logic, we design a prompt P_x with the following structure:

> ***Instruction:*** *[Input x];*
> ***Requirement:*** *Output the logical reasoning sketch step-by-step. At the end, select the result from [Choice_1], [Choice_2], ..., [Choice_n] with confidence Very Low/ Low/ Medium/ High/ Very High.*

With prompt P_x, the process of generating reasoning logic r_s and result y_s for M_s under input x can be expressed as:

$$\Gamma\left(x, P_x\right) : \left[r_s, y_s\right] = M_s\left(x, P_x\right). \tag{1}$$

In addition, we can obtain the logit array corresponding to the result token after the SLM completes reasoning in all transformer layers and undergoes regularization by RMSNorm, which holds all the possible values of the result output token and its probability. It's important to note that we adjust the SLM's temperature and sampling rate to avoid overly confident models that result in sharp token probability distributions. The set of possible results is denoted as $\mathbf{C} = \left\{C_1^x, C_2^x, ..., C_n^x\right\}$, where n is the number of possible results. We encode C_i to the corresponding token id $C_i^x\_ID$ by the tokenizer and query the logit array to map the probability value $C_i^x\_TP$. The Softmax function is then used to normalize the probability values of all results to obtain the normalized probability $C_i^x\_NP$ of each possible choice being selected. The normalized probability can be calculated as:

$$C_{i\_NP}^x = \frac{e^{C_{i\_TP}^x}}{\sum_{j=1}^{n} e^{C_{j\_TP}^x}}. \tag{2}$$

Then, the highest probability among them is defined as the confidence $\mathcal{L}$ of the SLM reasoning result y_s with input x, which can be represented as:

$$\mathcal{L}\left(x : y_s\right) = \max\left(C_{1\_NP}^x, C_{2\_NP}^x, ..., C_{n\_NP}^x\right). \tag{3}$$

The confidence score reflects the SLM's certainty about the generated results. A low confidence typically indicates high task complexity, suggesting that the SLM may struggle to accurately capture complex reasoning logic. However, considering that the model may exhibit unstable and overly confident behavior during inference, we also provide a confidence level prompt during SLM reasoning, such as "Please provide your confidence in the answer only as one of Very Low/ Low/ Medium/ High/ Very High". Each level is mapped to a different confidence value range, which serves as the basis for the final task assignment. Tasks with a confidence score below a threshold will be reasoned through the collaboration of LLMs and SLMs. Therefore, setting a reasonable threshold γ to distinguish between simple and difficult tasks is crucial for both result accuracy and service cost optimization. Based on experience, we set an initial threshold and aim to explore the optimal cost-performance balance in our experiments.

3.3 Prompt Tuning-based Reasoning Optimization

Adaptive Prompt Compression. After task assignment, complex tasks are uploaded to LLMs for reasoning, which often have long contexts and complex logic. Therefore, prompt compression can effectively reduce the cost. Considering the impact of different compression levels on the final reasoning accuracy, we design the adaptive prompt compression method related to the confidence $\mathcal{L}\left(x : y_s\right)$.

Based on the analysis in the previous section, the confidence interval of the SLM's reasoning results is $[1/n, r)$, and the compression rate must have both a lower limit l_{dn} and an upper limit l_{up}. Following the principle that a higher confidence leads to a lower compression rate, we map the confidence interval to the compression rate interval $\left(l_{dn}, l_{up}\right]$ using a linear mapping. The specific mapping formula for the compression rate $\mathcal{Z}$ is as follows:

$$\mathcal{Z}\left(\mathcal{L}\right) = l_{up} - \left(\frac{l_{up} - l_{dn}}{r - 1/n}\right) * \left(\mathcal{L} - \frac{1}{n}\right). \tag{4}$$

Then, another designed prompt $P_x^{\mathcal{Z}}$ is used to guide the SLM to adaptively compress the original problem according to the compression rate, avoiding wasted cost or severe accuracy loss caused by a fixed compression rate. The structure of prompt $P_x^{\mathcal{Z}}$ is as follows:

> ***Instruction:*** *[Input x];*
> ***Requirement:*** *Compress the problem without changing the core logic and the reference compression rate is* $[\mathcal{Z}\left(\mathcal{L}\right)]$.

It should be noted that excessive compression may remove essential contextual cues, leading to a significant drop in reasoning accuracy once the compression rate exceeds a task-dependent threshold. For context-heavy tasks, the adaptive compression range should be set more conservatively to preserve key information. In the current framework, the compression bounds l_{dn} and l_{up} are empirically determined; their optimal values may vary across tasks and models. Future work will investigate automatic adjustment strategies, such as Bayesian optimization, to dynamically adapt these parameters.

Information Gain for LLMs. Considering the black-box nature of API calls to LLMs, in order to improve the reasoning accuracy, we propose a dynamic exclusion mechanism based on information gain as illustrated in Fig. 4, which optimizes the reasoning process of LLMs through the low-confidence option information provided by the SLM.

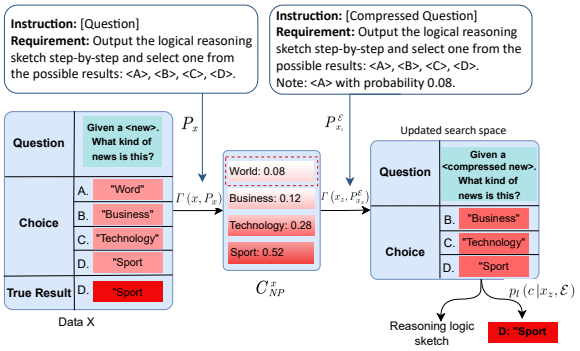


Figure 4: The illustration of information gain for LLMs.

After obtaining the probability distributions of all possible results of M_s reasoning about the input x, we filter the K choices with the lowest probability to form the exclusion set $\mathcal{E} = \{C_{e1}, ..., C_{eK}\}$, where $1 \leqslant K < n$. The exclusion set $\mathcal{E}$ and its probability information are injected into the prompt template of the LLMs to construct an enhanced prompt $P^{\mathcal{E}}_{x_z}$. The prompt structure can be represented as:

> ***Instruction:*** *[Compressed input x_z];*
> ***Requirement:*** *Output the logical reasoning sketch step-by-step and select one from the possible results: [Choice_1], [Choice_2], ..., [Choice_n].*
> ***Note:*** *[Choice_e1] with probability [$C^x_{e1\_NP}$],..., [Choice_eK] with probability [$C^x_{eK\_NP}$].*

With prompt $P^{\mathcal{E}}_x$, the process of generating reasoning logic r_l and result y_l for the LLM M_l can be expressed as:

$$\Gamma\left(x_z, P^{\mathcal{E}}_{x_z}\right) : [r_l, y_l] \;=\; M_l\left(x_z, P^{\mathcal{E}}_{x_z}\right). \tag{5}$$

To demonstrate the effectiveness of information gain, we perform a simple theoretical analysis based on information theory. The original conditional entropy of LLMs without exclusion information can be formulated as:

$$H_l\left(C\,|x_z\right) = -\sum_{c\in C} p_l\left(c\,|x_z\right)\log p_l\left(c\,|x_z\right). \tag{6}$$

After injecting exclusion information, the conditional entropy can be expressed as:

$$H_l\left(C\,|x_z, \mathcal{E}\right) = -\sum_{c\in C\setminus\mathcal{E}} p_l\left(c\,|x_z, \mathcal{E}\right)\log p_l\left(c\,|x_z, \mathcal{E}\right). \tag{7}$$

When the LLM excludes the low probability options, the probability distributions of the remaining options will also be updated to:

$$p_l\left(c\,|x_z, \mathcal{E}\right) = \frac{p_l\left(c\,|x_z\right)}{1-\sum_{c_e\in C\setminus\mathcal{E}} p_l\left(c_e\,|x_z\right)}, \forall c \in C\setminus\mathcal{E}. \tag{8}$$

This operation concentrates the probability mass towards the high confidence region. For a discrete random variable C, when some low-probability events are excluded, the entropy of the remaining distribution satisfies:

$$H\left(\frac{p\left(c\right)}{1-\sum p\left(c_e\right)}\right) < H\left(p\left(c\right)\right),\ if \sum p\left(c_e\right) > 0, \tag{9}$$

which is guaranteed by the concavity of Shannon entropy [1]. Therefore, according to the definition of information gain, we can derive the information gain as follows:

$$IG\left(C\,|x_z\,, \mathcal{E}\right) = H_l\left(C\,|x_z\right) - H_l\left(C\,|x_z, \mathcal{E}\right) \geqslant 0. \tag{10}$$

That is, by injecting the information of $\mathcal{E}$, the reasoning uncertainty of the LLMs is reduced and the information gain is improved.

Logic Alignment for SLMs. After the LLM receives the enhanced information from SLMs and performs the reasoning task, it outputs only a logic sketch, denoted as r_{sketch} and a final reasoning result y_l as required by the prompt for cost consideration. The complete reasoning logic is supplemented by SLMs. To ensure the consistency of the reasoning logic of both models and the coherence of the final output logic of the SLM, we propose a logic alignment method for logic restoration of SLMs. The logic sketch r_{sketch} is first decoded into token ids, and then the id sequences are converted into feature vectors $v_{r_{sketch}}$ by embedding. The feature vectors of the SLM reasoning logic can be obtained directly from the output of the last FFN layer that is not normalized at the time of inference. Mean pooling is used to generate sentence-level vector representations v_{r_s}. The cosine similarity between r_{sketch} and the r_s is computed using the generated vector representation as follows:

$$cos\_sim\left(v_{r_{sketch}}, v_{r_s}\right) = \frac{v_{r_{sketch}} \cdot v_{r_s}}{\left\|v_{r_{sketch}}\right\|\left\|v_{r_s}\right\|}. \tag{11}$$

Cosine similarity is adopted for logic alignment as it removes the effect of vector magnitude and focuses on semantic direction consistency, providing a robust and efficient measure for comparing reasoning logics from different models. Based on the cosine similarity, the reasoning logic sketch is selectively combined with the specific logic in order to leverage the strong generalization ability of LLMs with the local detail mastery of SLMs. Eventually, the aligned logic sketches and the result are prompted to the SLM to output the complete reasoning logic r and final results y.

Algorithm 1: A Cost-aware Approach for Collaborating LLMs and SLMs

Input: An instruction x.
Output: Reasoning logit r, result y.

1 **Initialize:** Initialize hyperparameters γ, l_{up}, l_{dn};
2 Construct prompt P_x with x;
3 Encode P_x with a tokenizer;
4 M_s performs reasoning to get the reasoning logic r_s and result y_s according to (1);
5 Intercept logits of the last token output by M_s;
6 **for** *each result* $C_i^x \in \boldsymbol{C}$ **do**
7 Encode C_i^x to token id $C_i^x\_ID$;
8 Map $C_i^x\_ID$ to token probability $C_i^x\_TP$;
9 **end**
10 **for** $i = 1$ *to* n **do**
11 Calculate the normalized probability $C_i^x\_NP$ according to (2);
12 **end**
13 Calculate the confidence $\mathcal{L}$ according to (3);
14 **if** $\mathcal{L} \geqslant \gamma$ **then**
15 r$\leftarrow r_s, y \leftarrow y_s$;
16 **else**
17 Calculate the compression rate $\mathcal{Z}$ according to (4);
18 M_s compresses x to x_z with prompt $P_x^{\mathcal{Z}}$;
19 Select K results with the lowest probability to form exclusion set $\mathcal{E}$;
20 M_l performs reasoning to get the reasoning logic sketch r_{sketch} and result y_l according to (5);
21 Transform r_{sketch} and r_s to feature vectors $v_{r_{sketch}}$ and v_{r_s}.
22 Compute cosine similarity according to (11)
23 Align logic sketch and prompt the SLM to generate final reasoning logic r and result y;
24 **end**
25 Return reasoning logic r and result y.

3.4 Summary

The workflow of our proposal is presented in Algorithm 1. Specifically, the original input is first added to the prompt, and then the prompt is encoded. Based on the encoded prompt, a SLM performs reasoning and outputs the reasoning logic and results (Lines 1-4). The logits of the last token in the reasoning results are used to calculate the probability of each reasoning result, which is subsequently normalized to derive the confidence of the reasoning results (Lines 5-13). If the confidence exceeds a predefined threshold, the reasoning logic and results are output directly (Lines 14-15). Otherwise, the original question is compressed and combined with the K lowest probability options, which are then embedded into the prompt to guide the LLM in generating a sketch of the reasoning logic and results (Lines 16-20). Finally, the complete logic from the SLM is semantically aligned with the sketch generated by the LLM to obtain a fused reasoning logic sketch, which is embedded into the prompt to guide the small model in refining the logic and outputting the final results (Lines 21-25).

4 Experimental Analysis

4.1 Experiment Setup

Datasets and Evaluation Metrics

We experiment on three open-source reasoning datasets, including:

- Amazon Reviews [20] is a natural language processing dataset commonly used for sentiment analysis that contains information about user reviews and ratings of various products. The elements "reviewText" and "overall" are extracted to build our dataset with 3-type sentiment.
- AGNews [34] is a standard dataset widely used for text classification and contains news articles in four categories: world, sports, business and technology.
- ANLI [21] is a challenging natural language reasoning dataset containing more counterexamples and complex sentence structures. The elements "premise", "hypothesis" and "label" are extracted from the original dataset.

Baselines and Prompt Methods

- CoT [28]. The method prompts the language model to reason out the results step by step. We use separate small and large models to serve as baselines for reasoning accuracy and service cost, respectively.
- Data Shunt [6]. The method prompts SLMs to choose to perform the reasoning task locally or upload it to LLMs based on the task complexity, which is used as a baseline to evaluate the effectiveness of our approach in reducing costs at the token level.

Model Selection

For our experiments, we chose one model from each of the Llama and Qwen series for fine-tuning and quantization to construct the experimental SLMs, including fine-tuned Llama3-8B [17] with 4-bit quantization and fine-tuned Qwen-14B [3] with 4-bit quantization. GPT4o-mini, the cheapest model of the current OpenAI platform, is accessed as a LLM by API calls. In addition, the DeepSeek-V3 is accessed as a third evaluator to score the reasoning logic.

Evaluation Metrics and Prompt Method

The three main metrics used in our experiments are reasoning accuracy Acc, reasoning logic score S_{core}, and reasoning cost C_{Api}. The three datasets used in the experiments are clearly labeled, so the accuracy of the reasoning results can be directly calculated. For the generated reasoning logic, we uniformly deployed Deepseek-V3 to score the generated content from four perspectives, with a total score of 10 points and 2.5 points for each point, including consistency with the results, output completeness, logical tightness, and level of detail. To ensure fairness, outputs from different models on the same set of experiments were evaluated using DeepSeek-V3 within the same session and time window, with consistent scoring criteria explicitly instructed to the model. In addition, we evaluate the service cost by calculating the cost of API calls generated per 1000 tasks processed.

4.2 Performance Evaluation

In this section, we evaluate our methods on AGNews, ANLI, and Amazon Review datasets, using reasoning accuracy Acc, reasoning logic score S_{core}, and reasoning cost C_{Api} as key evaluation metrics.

Table 1: Performance and cost evaluation.

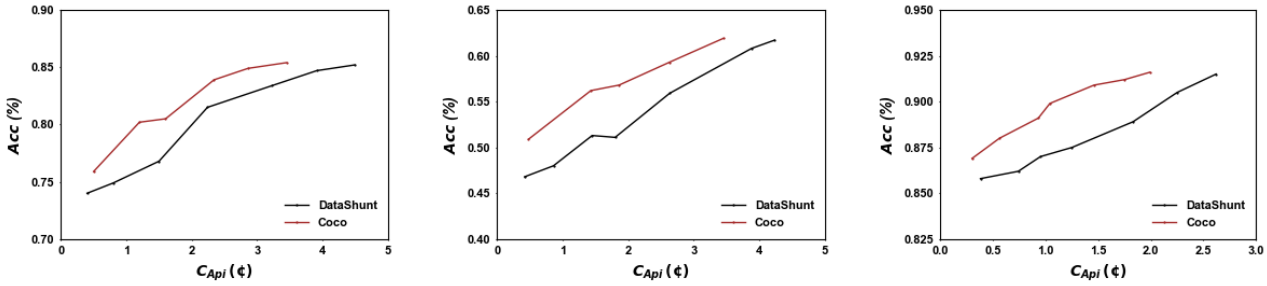
Models	Methods	AGNews			ANLI			Amazon Review		
		Acc	S_{core}	C_{Api}	Acc	S_{core}	C_{Api}	Acc	S_{core}	C_{Api}
GPT4o-mini	CoT (LLM)	86.80%	9.3	4.59¢	65.80%	9.0	4.85¢	92.30%	9.5	2.81¢
Llama3-8B-FQ	CoT (SLM-FQ)	73.62%	8.0	N/A	46.40%	7.4	N/A	85.24%	8.7	N/A
	DataShunt (L+S$_{FQ}$)	**84.67%**	8.3	3.91¢	**60.82%**	8.2	3.88¢	90.47%	9.0	2.25¢
	Ours (L+S$_{FQ}$)	83.91%	8.8	2.34¢	59.33%	8.2	2.63¢	**90.72%**	9.2	1.46¢
	Improvement	0.90%↓	6.02% ↑	40.15%↓	2.45%↓	0%	32.31%↓	0.28% ↑	2.22% ↑	35.11% ↓
Qwen-14B-FQ	CoT (SLM-FQ)	77.93%	8.8	N/A	61.66%	8.4	N/A	88.58%	9.2	N/A
	DataShunt (L+S$_{FQ}$)	**85.75%**	8.9	3.62¢	65.24%	8.4	3.39¢	**92.42%**	9.3	1.91¢
	Ours (L+S$_{FQ}$)	84.86%	9.2	2.10¢	**65.75%**	9.0	2.44¢	91.57%	9.6	1.32¢
	Improvement	1.04%↓	3.37% ↑	41.99%↓	0.78% ↑	7.14% ↑	28.02%↓	0.92%↓	3.22% ↑	30.89%↓

The baseline methods include the chain-of-thought cueing method and the currently popular model co-division method, respectively. Among them, CoT (LLM) and CoT (SLM), which completely use large and small models to process the task, respectively, are used as our evaluation criteria baseline. The Datashunt, as a representative of model collaboration methods, is used to evaluate the effectiveness of our methods in token-level cost optimization, as well as the reasonableness of the reasoning logic. We conducted experiments based on Llama-3-8B and Qwen-14B respectively, where both models were fine-tuned and quantized to ensure basic inference performance and speed, and the results are shown in Table 1. Bolding indicates the optimal inference accuracy, while underlining is the baseline value against which improvement is aligned.

In the AGNews task, our method achieved an accuracy of 83.91% on Llama3-8B-FQ and 84.86% on Qwen-14B-FQ, which is slightly lower than the DataShunt method by less than 1%. This minor decrease can be attributed to our cost-aware reasoning process, which selectively compresses and restructures the input, potentially omitting some less critical but still beneficial contextual details. However, despite this slight reduction in accuracy, our method demonstrated improvements in reasoning quality, as indicated by a reasoning score of 9.2 on Qwen-14B-FQ, outperforming DataShunt's 8.9. More importantly, our approach significantly reduced API costs, cutting them by over 40% on both models. This confirms that our confidence-based task allocation strategy effectively shifts simpler reasoning components to the small model while minimizing unnecessary LLM token generation, thereby optimizing overall cost efficiency.

For the more challenging ANLI task, which involves complex logical reasoning, our method demonstrated strong improvements in both accuracy and reasoning score. On Qwen-14B-FQ, our accuracy reached 65.75%, outperforming DataShunt by more than half a percentage point. This gain suggests that our reasoning optimization strategy successfully refines and compresses input prompts without compromising core logical structures. On Llama3-8B-FQ, our method achieved an accuracy of 59.33%, slightly lower than DataShunt, which can be explained by the inherent difficulty of ANLI, where even minor changes in input representation can influence the final reasoning trajectory. However, our method led to a notable improvement in reasoning score, with Qwen-14B-FQ increasing from 8.4 to 9.0, reflecting a more structured and logically sound reasoning process. In terms of cost efficiency, API expenses were reduced by approximately 32% on Llama3-8B-FQ and 28% on Qwen-14B-FQ. This further validates our method's ability to enhance reasoning capabilities while reducing reliance on expensive LLM inference.

In the Amazon Review dataset, which focuses on sentiment analysis and review classification, our method maintained competitive accuracy, reaching 90.72% on Llama3-8B-FQ and 91.57% on Qwen-14B-FQ. Compared to DataShunt, the accuracy was slightly lower by less than one percent. The minor reduction can be attributed to the fact that sentiment-based tasks often rely on subtle linguistic nuances, which may be slightly affected by our input compression strategy. However, the reasoning score showed significant improvement, reaching 9.6 on Qwen-14B-FQ, surpassing DataShunt's 9.3. This suggests that our method enhances interpretability and response coherence, likely due to its structured prompt tuning approach. Additionally, API costs were substantially reduced by approximately 35% on Llama3-8B-FQ and 31% on Qwen-14B-FQ, reinforcing the economic benefits of our method in real-world applications where API call expenses are a major concern. Overall, our method demonstrates a well-balanced trade-off between accuracy, reasoning logic quality, and service cost efficiency. While accuracy is marginally lower in some tasks due to the controlled compression of token interactions, the improvements in reasoning score indicate that our method maintains, or even enhances, logical coherence and interpretability. Most importantly, API costs were consistently reduced across all datasets, with savings ranging from 28% to 42%. This makes our approach particularly valuable for commercial applications that require intelligent reasoning while operating under strict cost constraints. The results validate our



(a) AGNews (b) ANLI (c) Amazon Review

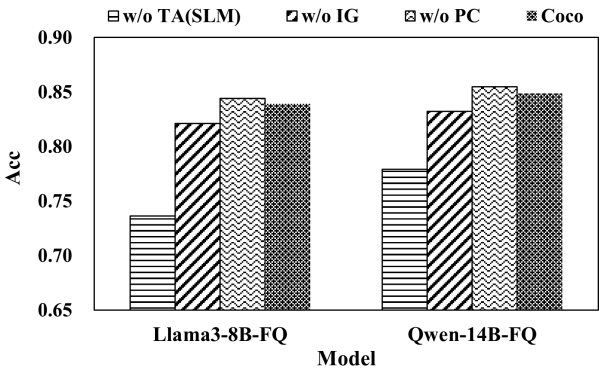
Figure 5: $Acc - C_{Api}$ trade–off curves.

confidence-based task assignment and prompt tuning-based reasoning optimization, proving that small models can effectively assist LLMs by handling lower-confidence tasks locally while leveraging LLMs only for necessary high-complexity reasoning, ultimately improving both efficiency and affordability in task reasoning.

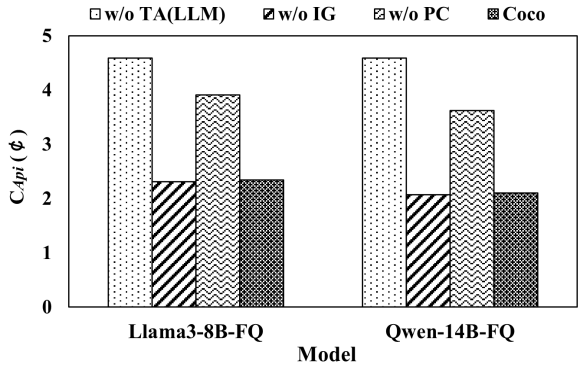
4.3 Trade-off between Accuracy and Cost

In this section, we delve into the detailed relationship between cost and inference accuracy to determine upper bounds on accuracy under different budget constraints. We still deploy DataShunt as a baseline to adjust the proportion of tasks processed by LLM versus the amount of single token interactions by simultaneously adjusting the confidence thresholds of both methods as well as the upper and lower bounds of compression rates in our approach. We test our method on three benchmarks, plotting the trade-off curves for accuracy versus API cost.

As illustrated in Fig. 5, trade-off curves of Coco consistently remain above the DataShunt baseline curves. Under identical accuracy levels, the token-related costs of Coco are significantly lower compared to DataShunt, highlighting the superior efficiency of our proposed task assignment and adaptive prompt compression methods. Moreover, with increasing budget constraints, Coco consistently achieves higher upper-bound performance, demonstrating its scalability and effectiveness in optimizing the trade-off between accuracy and operational costs. Additionally, our results suggest potential avenues for further performance improvement. Specifically, by raising the confidence threshold for task assignment or tightening the adaptive compression rate range, it is possible to further boost accuracy at the expense of slightly increased costs. These adjustments provide a flexible mechanism for fine-tuning performance according to specific operational or budgetary requirements, further validating the effectiveness and efficiency of the proposed collaboration between LLMs and SLMs.



(a) Impact of core components on accuracy.



(b) Impact of core components on cost.

Figure 6: Result of ablation study on AGNews.

4.4 Ablation Study

The ablation study conducted on the AGNews dataset evaluates the impact of Coco's key modules, task assignment (TA), information gain (IG), and adaptive prompt compression (PC), on its overall performance. Specifically, experiments were performed to assess reasoning accuracy A_{cc} and service cost C_{Api}, using two SLMs: Llama3-8B-FQ and Qwen-14B-FQ. By comparing Coco with its ablated versions (w/o TA, w/o IG, and w/o PC), insights were gained into the contribution of each key module and their effectiveness in improving reasoning accuracy and reducing costs.

As depicted in Fig. 6(a), completely removing the TA module and processing all tasks solely on the SLM significantly reduces model

accuracy. Conversely, processing entirely on the LLM without the TA module achieves slightly higher accuracy than the collaborative approach, indicating that while the collaborative method slightly sacrifices accuracy compared to purely LLM-based processing, it achieves substantially higher accuracy compared to purely SLM-based processing. The omission of the IG module reduces model accuracy, clearly indicating its positive contribution to improving inference capabilities in the large model. Removing the PC module, which is designed to compress cues, results in a slight decrease in accuracy. In fact, the effect of the PC module on the reasoning accuracy is usually influenced by the redundancy of the samples as well as the complexity of the context, which can play a positive role in accuracy improvement by further training of the SLM.

Fig. 6(b) illustrates the impact of each module on processing costs, specifically measured by API call cost. Exclusively relying on the LLM for task processing significantly increases costs, confirming that the task partitioning module effectively reduces unnecessary LLM usage. Furthermore, removing the PC module notably raises the overall processing cost, underscoring its crucial role in compressing inputs and prompting concise model outputs to minimize token-level expenses. Meanwhile, the inclusion of the IG module has a negligible effect on inference costs. Ultimately, Coco, integrating all components, achieves the lowest processing cost, validating that the combined modules effectively reduce token interactions without substantial loss in accuracy.

Overall, this ablation study demonstrates the distinct and combined contributions of the core components within Coco. It highlights that the IG module significantly enhances inference accuracy, while the PC module, despite slightly impacting performance due to prompt compression, effectively removes redundancy and reduces costs. Thus, integrating these components demonstrates a practical balance between accuracy and cost-efficiency, emphasizing the effectiveness of the proposed collaborative approach between LLMs and SLMs.

5 Conclusion

In this paper, we propose Coco, a cost-aware approach for collaborating LLMs and SLMs. Our method utilizes a confidence-based task assignment strategy, effectively reducing reliance on expensive LLM API calls by intelligently delegating tasks to SLMs. Considering that tasks uploaded to LLMs for processing are often contextually complex and difficult, prompt tuning-based reasoning optimization techniques are further introduced to reduce token interactions while maintaining reasoning accuracy. Experimental results on three diverse open-source datasets showed that Coco significantly reduces API call costs by up to 42% compared to baseline methods while maintaining competitive accuracy and enhancing the quality and interpretability of reasoning logic. Future work will focus on exploring reinforcement learning approaches to dynamically optimize task assignment and prompt compression strategies. Additionally, further research on privacy-aware collaboration methods is necessary to protect contextual privacy during personalized collaborative reasoning.

Acknowledgment

This research is supported by the National Key Research and Development Program of China under Grant No. 2024YFE0204500, and in part by the National Natural Science Foundation of China under Grant No.92267104.

GenAI disclosure statement

ChatGPT-4o was solely used to check spelling and improve grammar during the writing of this paper.

References

[1] Aqib Ali, Sania Anam, and Muhammad Munawar Ahmed. 2023. Shannon entropy in artificial intelligence and its applications based on information theory. *Journal of Applied and Emerging Sciences* 13, 1 (2023), 09–17.

[2] Avinash Anand, Ashwin R Nair, Kritarth Prasad, Vrinda Narayan, Naman Lal, Debanjan Mahata, Yaman K Singla, and Rajiv Ratn Shah. 2024. Advances in Citation Text Generation: Leveraging Multi-Source Seq2Seq Models and Large Language Models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 56–64.

[3] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[4] Benjamin Bergner, Andrii Skliar, Amelie Royer, Tijmen Blankevoort, Yuki Asano, and Babak Ehteshami Bejnordi. 2024. Think big, generate quick: Llm-to-slm for fast autoregressive decoding. *International Conference on Machine Learning* (2024).

[5] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology* 15, 3 (2024), 1–45.

[6] Dong Chen, Yueting Zhuang, Shuo Zhang, Jinfeng Liu, Su Dong, and Siliang Tang. 2024. Data shunt: Collaboration of small and large models for lower costs and better performance. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 11249–11257.

[7] Liangyu Chen, Bo Li, Sheng Shen, Jingkang Yang, Chunyuan Li, Kurt Keutzer, Trevor Darrell, and Ziwei Liu. 2023. Large language models are visual reasoning coordinators. *Advances in Neural Information Processing Systems* 36 (2023), 70115–70140.

[8] Yao Fu, Hao Peng, Litu Ou, Ashish Sabharwal, and Tushar Khot. 2023. Specializing smaller language models towards multi-step reasoning. In *International Conference on Machine Learning*. PMLR, 10421–10430.

[9] Zixu Hao, Huiqiang Jiang, Shiqi Jiang, Ju Ren, and Ting Cao. 2024. Hybrid slm and llm for edge-cloud collaborative inference. In *Proceedings of the Workshop on Edge and Mobile Foundation Models*. 36–41.

[10] Yichen Jiang, Marco Vecchio, Mohit Bansal, and Anders Johannsen. 2024. Hierarchical and dynamic prompt compression for efficient zero-shot API usage. In *Findings of the Association for Computational Linguistics: EACL 2024*. 2162–2174.

[11] Hunter Lang, Monica N Agrawal, Yoon Kim, and David Sontag. 2022. Co-training improves prompt-based learning for large language models. In *International Conference on Machine Learning*. PMLR, 11985–12003.

[12] Junyi Li, Tianyi Tang, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2024. Pre-trained language models for text generation: A survey. *Comput. Surveys* 56, 9 (2024), 1–39.

[13] Liunian Harold Li, Jack Hessel, Youngjae Yu, Xiang Ren, Kai-Wei Chang, and Yejin Choi. 2023. Symbolic chain-of-thought distillation: Small models can also" think" step-by-step. *Findings of the Association for Computational Linguistics* (2023).

[14] Jinggui Liang, Lizi Liao, Hao Fei, and Jing Jiang. 2024. Synergizing large language models and pre-trained smaller models for conversational intent discovery. In *Findings of the Association for Computational Linguistics ACL 2024*. 14133–14147.

[15] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2023. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM computing surveys* 55, 9 (2023), 1–35.

[16] Yubo Ma, Yixin Cao, YongChing Hong, and Aixin Sun. 2023. Large language model is not a good few-shot information extractor, but a good reranker for hard samples! *Conference on Empirical Methods in Natural Language Processing* (2023).

[17] AI Meta. 2024. Introducing meta llama 3: The most capable openly available llm to date. *Meta AI* 2, 5 (2024), 6.

[18] Bonan Min, Hayley Ross, Elior Sulem, Amir Pouran Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. 2023. Recent advances in natural language processing via large pre-trained language models: A survey. *Comput. Surveys* 56, 2 (2023), 1–40.

[19] Alliot Nagle, Adway Girish, Marco Bondaschi, Michael Gastpar, Ashok Vardhan Makkuva, and Hyeji Kim. 2025. Fundamental limits of prompt compression: A rate-distortion framework for black-box language models. *Advances in Neural Information Processing Systems* 37 (2025), 94934–94970.

[20] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*. 188–197.

[21] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. 2020. Adversarial NLI: A New Benchmark for Natural Language Understanding. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics.

[22] Samet Oymak, Ankit Singh Rawat, Mahdi Soltanolkotabi, and Christos Thrampoulidis. 2023. On the role of attention in prompt-tuning. In *International Conference on Machine Learning*. PMLR, 26724–26768.

[23] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2025. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems* 37 (2025), 126544–126565.

[24] Hendrik Strobelt, Albert Webson, Victor Sanh, Benjamin Hoover, Johanna Beyer, Hanspeter Pfister, and Alexander M Rush. 2022. Interactive and visual prompt engineering for ad-hoc task adaptation with large language models. *IEEE transactions on visualization and computer graphics* 29, 1 (2022), 1146–1156.

[25] Mahiro Ukai, Shuhei Kurita, Atsushi Hashimoto, Yoshitaka Ushiku, and Nakamasa Inoue. 2024. AdaCoder: Adaptive Prompt Compression for Programmatic Visual Question Answering. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 9234–9243.

[26] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R Lyu. 2022. No more fine-tuning? an experimental evaluation of prompt tuning in code intelligence. In *Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering*. 382–394.

[27] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations* (2023).

[28] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.

[29] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2024. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems* 36 (2024).

[30] Jiali Zeng, Fandong Meng, Yongjing Yin, and Jie Zhou. 2024. Teaching large language models to translate with comparison. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19488–19496.

[31] Congzhi Zhang, Linhai Zhang, Jialong Wu, Yulan He, and Deyu Zhou. 2025. Causal prompting: Debiasing large language model prompting based on front-door adjustment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 25842–25850.

[32] Kaiyan Zhang, Jianyu Wang, Ermo Hua, Biqing Qi, Ning Ding, and Bowen Zhou. 2024. Cogenesis: A framework collaborating large and small language models for secure context-aware instruction following. *Findings of the Association for Computational Linguistics* (2024).

[33] Xuan Zhang, Chao Du, Tianyu Pang, Qian Liu, Wei Gao, and Min Lin. 2024. Chain of Preference Optimization: Improving Chain-of-Thought Reasoning in LLMs. *arXiv preprint arXiv:2406.09136* (2024).

[34] Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. *Advances in neural information processing systems* 28 (2015).

[35] Beier Zhu, Yulei Niu, Yucheng Han, Yue Wu, and Hanwang Zhang. 2023. Prompt-aligned gradient for prompt tuning. In *Proceedings of the IEEE/CVF international conference on computer vision*. 15659–15669.